

AKTU B.Tech Cheat Sheet — Software Engineering (BCS 601)

Semester/Year: 6th Sem / 3rd Year **Branch:** CSE **Target:** 90–100% marks

(Subject code was KCS601 under the old syllabus; changed to BCS601 from 2024-25 onwards.)

1. AKTU EXAM PATTERN MAPPING

AKTU pattern for BCS 601 (70 marks / 3 hrs):

- **Section A** — 7×2 marks (compulsory, short answer, covers all 5 units) = **14**
- **Section B** — 3 out of 5×7 marks (medium answer, any 3 attempts) = **21**
- **Section C** — 5×7 marks (Q3–Q7, one from each unit with internal choice "any 1 out of 2") = **35**
- **Total = 70 marks**

Structure of Section C: Q3 (Unit I) · Q4 (Unit II) · Q5 (Unit III) · Q6 (Unit IV) · Q7 (Unit V), each with 2 options, pick 1. Every unit **MUST** be attempted.

TREND ALERT (based on actual 2023-24 & 2024-25 papers)

- **Subject code changed:** KCS601 (old, 100 marks) → BCS601 (new, 70 marks).
- **Heavy numerical shift in 2024-25.** 2023-24 had only one numerical (prime number BVA). 2024-25 has **three numerals in Section C alone** (CAF/FP with $\pm 35\%$, quadratic BVA+EP, CFG + cyclomatic complexity).
- **K-level cognitive tagging** (K1–K5) introduced in 2024-25. K4–K5 questions (Apply/Analyze/Evaluate) dominate Section C.
- **Repeated topics across both years:** SEI CMM vs ISO 9000, Cohesion & Coupling, Software Maintenance types, Re-engineering, Size metrics (LOC/FP).
- **Expected for 2025-26:** COCOMO numerical is overdue (both years asked it theoretically only — a calculation question is very likely). UML diagrams, DFD, and Halstead's metrics are all high-probability.

| Topic-to-Section Mapping + PYQ Frequency (2023-24 & 2024-25 actual)

Unit	Topic	Usual Section	Frequency	Tag
I	Software characteristics (wear-out, not manufactured)	A	Very High	MUST-DO
I	Software components	A + C	High	MUST-DO
I	SDLC models (Waterfall/Prototype/Spiral/Evolutionary)	C	Very High	MUST-DO
I	Software Quality Attributes	B + C	High	MUST-DO
I	Software crisis	A + C	High	MUST-DO
I	SE = "layered technology"	B	Medium	High-yield
II	SRS need + IEEE standard	A + C	High	MUST-DO
II	Requirement Elicitation techniques	B + C	High	MUST-DO
II	Feasibility Study (TOE)	B	High	MUST-DO
II	DFD (levels, notation)	B + C	Very High	MUST-DO
II	Use Case + template	A + C	High	MUST-DO
II	Decision Tables / ER Diagrams	B + C	Medium	High-yield
II	SEI CMM vs ISO 9000	C	Very High (both yrs)	MUST-DO
II	Error Index / SQA statistical procedure	B	Medium	—
II	UML diagrams (9 types)	C	High	MUST-DO
III	Cohesion & Coupling (all types)	C	Very High (both yrs)	MUST-DO
III	Cyclomatic Complexity (3 formulas)	B + C	Very High	MUST-DO
III	Function Point + CAF	B + C	Very High	MUST-DO
III	Halstead's Software Science	C	High	MUST-DO
III	Function-Oriented vs Object-Oriented Design	B + C	Medium	—
III	Modularization / Design Structure Chart	B	Medium	—
III	Software design attributes	C	High	MUST-DO
IV	BVA + Equivalence Partitioning numerical	C	Very High (both yrs)	MUST-DO
IV	White Box (Structural) vs Black Box	B + C	High	MUST-DO
IV	Unit/Integration/System testing	B + C	Very High	MUST-DO
IV	Stubs and Drivers	A + B	High	MUST-DO
IV	Alpha, Beta, Mutation, Regression	A + B	High	MUST-DO
IV	Code Inspection/Walkthrough/FTR	B + C	High	MUST-DO
IV	Error, Fault, Failure	A	High	MUST-DO
V	Maintenance types (P/C/P) + cost	B + C	Very High (both yrs)	MUST-DO
V	COCOMO model + Person-Month	C	High	MUST-DO
V	Reverse Engineering vs Re-engineering	A + C	Very High (both yrs)	MUST-DO
V	Software Configuration Management	B	Medium	High-yield
V	CASE tools	C	Medium	High-yield
V	Risk management (RMMM)	A + B	High	MUST-DO

📌 **Priority order for last-week prep:** Cyclomatic Complexity → BVA/EP → Function Point/CAF → COCOMO → Cohesion/Coupling → SEI CMM vs ISO → DFD → Maintenance types → SDLC models.

2. UNIT-WISE QUICK REVISION NOTES

UNIT I — Introduction to Software Engineering & SDLC Models

(a) Core Definitions

- **Software Engineering:** Systematic, disciplined, quantifiable application of engineering approaches to design, develop, operate, and maintain software (IEEE definition).
- **Software:** (1) Computer programs, (2) documentation, (3) data structures/configurations needed to operate the programs.
- **Software Characteristics:**
 - *Software is developed/engineered, not manufactured in classical sense.*
 - *Software doesn't wear out* (but deteriorates due to changes — "bathtub curve" replaced by "change accumulation curve").
 - *Most software is custom-built* (not assembled from components).
 - *Software is logical, not physical.*
 - *Software is intangible and invisible.*
- **Software Components:** Programs + Documentation + Operating procedures (triad).
- **Software Crisis:** Inability of developers to deliver software on time, within budget, with required quality — symptoms: cost overrun, schedule slip, poor quality, high maintenance.
- **Software Quality Attributes (McCall's Model — 11 factors, 3 categories):**
 - *Product Operation:* Correctness, Reliability, Efficiency, Integrity, Usability
 - *Product Revision:* Maintainability, Flexibility, Testability
 - *Product Transition:* Portability, Reusability, Interoperability

(b) SE vs Conventional Engineering

Aspect	Conventional	Software
Product	Tangible	Intangible
Wear-out	Yes	No (deteriorates logically)
Standard components	Yes	Rare
Duplication cost	High	Near zero
Maintenance	Replace parts	Modify code

(c) SDLC Models — Comparison Table ★ [MUST-MEMORIZE]

Model	Key Feature	Best For	Limitation
Waterfall	Linear sequential; no backtracking	Well-understood stable requirements	Late defect detection; inflexible
Prototype	Build throwaway/evolutionary prototype first	Unclear requirements	Customer may want prototype as final; poor documentation
Spiral	Iterative + risk analysis in each spiral	Large, high-risk projects	Complex, expensive; requires risk expertise
Evolutionary (Incremental)	Deliver in increments, core first	Partial requirements known	Integration issues; architecture debt
Iterative Enhancement	Repeated passes refining the product	Medium-sized well-understood systems	Planning overhead
RAD	Fast dev with component reuse	Modular, short projects	Needs experienced team; not for high-tech
V-Model	Waterfall with testing mirrored at each stage	Safety-critical	Rigid like waterfall

Waterfall phases (order): Requirement Analysis → System Design → Implementation → Integration & Testing → Deployment → Maintenance.

Spiral quadrants (4 per loop): (1) Determine objectives (2) **Identify & resolve risks** (3) Development & testing (4) Planning next iteration.

(d) Diagrams to Master

1. **Waterfall model** (6 cascading boxes with single arrows down, feedback arrows back up).
2. **Spiral model** (spiral with 4 quadrants, radial = cumulative cost, angle = progress).
3. **Prototype model** (Quick design → Build prototype → Customer evaluation → Refine → Final product).
4. **"Software Engineering is layered technology"** — 4 layers: Quality focus → Process → Methods → Tools.

(e) Frequently Asked Long Qs

- "Explain Software Characteristics." (2 marks + 7 marks) — 2023-24 & 2024-25
- "Compare SDLC models" / "Explain Spiral with diagram" — 2024-25 Q3(a)

- "Discuss Software Quality Attributes in detail" — 2023-24 Q3(a)
- "Why is there a crisis in software development?" — 2024-25 Q3(b)

(f) Solved PYQ-Style Problems

Q1 (2 marks). Justify: "Software does not wear out and is not manufactured." (2024-25 Q1.b)

Sol: *Doesn't wear out* — software is logical; physical degradation (corrosion, fatigue) doesn't apply. It deteriorates only when modifications introduce new defects ("change-induced deterioration"). *Not manufactured* — software is custom-engineered once, then replicated digitally at near-zero cost. No raw materials, assembly line, or quality-control-per-unit like physical products.

Q2 (2 marks). List components of software with brief description. (2024-25 Q1.a)

Sol:

1. **Programs** — executable code (source + object).
2. **Documentation** — SRS, design docs, user manuals, test cases.
3. **Operating procedures** — installation, configuration, backup/recovery scripts.

Q3 (7 marks). Explain the Spiral Model with diagram. (2024-25 Q3.a)

Sol:

- *Combines* iterative nature of prototyping + controlled and systematic aspects of waterfall + **risk analysis**.
- **4 quadrants per loop:**
 1. Objectives, Alternatives, Constraints
 2. **Risk Analysis** & prototyping
 3. Development & Verification (waterfall-like)
 4. Planning next phase
- *Radial axis* = cumulative cost; *angular axis* = progress through current spiral.
- Each loop produces a more refined product.
- **Selection criteria:** use when project is large, requirements evolve, risk is high, user involvement is high.
- **Advantages:** risk reduction, flexibility, incremental releases.
- **Disadvantages:** expensive, requires risk-analysis expertise, complex management.

■ UNIT II — SRS, Requirements Engineering & SQA

(a) Core Definitions

- **SRS (Software Requirements Specification):** Formal document describing WHAT the software should do (functional) and HOW well (non-functional), used as contract between client and developer.
- **Requirement Engineering:** Discipline concerned with eliciting, analyzing, documenting, validating, and managing software requirements.
- **Feasibility Study (TOE):** *Technical* (can it be built?), *Operational* (will it be used?), *Economic* (is it worth it?). Also *Legal*, *Schedule*.
- **Use Case:** Description of system behavior from user's perspective — captures a sequence of actions for achieving a specific user goal.
- **SQA (Software Quality Assurance):** Planned and systematic set of activities ensuring software quality standards are met throughout SDLC.
- **Verification:** "Are we building the product right?" — conformance to specs (static).
- **Validation:** "Are we building the right product?" — meeting user needs (dynamic).

(b) Requirement Engineering Process (6 phases) ★

1. **Inception** — establish basic understanding.
2. **Elicitation** — gather needs from stakeholders.
3. **Elaboration** — refine and expand requirements.
4. **Negotiation** — resolve conflicts, set priorities.
5. **Specification** — document in SRS.
6. **Validation** — review for completeness, correctness, consistency.
7. (*Management — ongoing throughout*)

Elicitation techniques: Interviews, Questionnaires, **Brainstorming**, Facilitated Application Specification Techniques (FAST), **Joint Application Development (JAD)**, Quality Function Deployment (QFD), Prototyping, Use Case modeling, Ethnography.

(c) IEEE 830 SRS Document Structure

1. Introduction (purpose, scope, definitions, references)
2. Overall description (product perspective, functions, user characteristics, constraints)
3. **Specific requirements** (functional, performance, interface, design, SQA)
4. Appendices

SRS Quality attributes: Correct, Complete, Consistent, Unambiguous, Verifiable, Modifiable, Traceable, Ranked, Understandable.

(d) Data Flow Diagram (DFD) — notation

Symbol	Meaning
Circle / Rounded rect	Process (transformation)
Arrow	Data flow
Open rectangle / Two parallel lines	Data store
Rectangle	External entity

Levels: Level 0 (Context diagram — one process) → Level 1 (main processes) → Level 2 (sub-processes) → ...

Rules: (1) No direct flow between stores or external entities. (2) Each process must have input AND output. (3) Names should be verb+noun (for processes) and noun (for stores/entities).

(e) ISO 9000 vs SEI-CMM ★ [APPEARED IN BOTH 2023-24 & 2024-25 — GUARANTEED]

Aspect		
Origin	International Org. for Standardization (generic)	Software Engineering Institute, Carnegie Mellon (software-specific)
Type	Quality management standard	Process maturity framework
Certification	Audited certification	Assessment (not certification)
Levels	Pass/Fail	5 maturity levels
Focus	Quality system	Process improvement
Scope	Any industry	Software development

SEI-CMM 5 Levels (mnemonic: "IRDMO"):

1. **Initial** — chaotic, ad-hoc, heroes.
2. **Repeatable** — basic project management, can repeat past successes.
3. **Defined** — standardized documented process for the organization.
4. **Managed** — quantitative process control with metrics.
5. **Optimizing** — continuous improvement via feedback and innovation.

(f) Frequently Asked Problems

Q1 (2 marks). What is Software Risk Analysis Management? (2024-25 Q1.c)

Sol: Systematic identification, assessment (probability × impact), prioritization, and mitigation of potential risks (technical, schedule, cost, operational) that could negatively affect a software project. Part of the **RMMM plan** (Risk Mitigation, Monitoring & Management).

Q2 (2 marks). Define Use Case + commonly accepted template. (2024-25 Q1.d)

Sol: *Use Case* = description of a set of scenarios tying together a common user goal. Template components:

- Use Case name
- Actor(s)
- Precondition
- Main flow / Basic course
- Alternative flows
- Postcondition
- Exceptions

Q3 (7 marks — DFD for Library Management Software). (2024-25 Q2.b)

Sol (outline):

- **Level 0 (Context):** one process "Library Management System" with external entities {User/Member, Librarian, Admin} and data flows {borrow request, issue confirmation, return notice, fines, reports}.
- **Level 1:** 4 processes — 1.0 Authenticate, 2.0 Issue/Return Books, 3.0 Generate Reports, 4.0 Database Maintenance. Data stores: D1 User_DB, D2 Books_DB, D3 Transactions_DB.
- **State Transition Diagram:** States = {Idle, Authenticating, AuthFailed, Authenticated, BookIssuing, BookReturning, ReportGenerating, Logout}; transitions triggered by user actions.

UNIT III — Software Design & Metrics

(a) Core Definitions

- **Software Design:** Process of defining architecture, components, interfaces, and data for a system to satisfy requirements.

- **Architectural Design:** High-level structure — decomposition into subsystems and their interactions.
- **Low-Level Design (Detailed Design):** Specifies individual modules — algorithms, data structures, interfaces.
- **Modularization:** Breaking system into manageable, independent modules.
- **Coupling:** Degree of interdependence BETWEEN modules. **LOW coupling = GOOD.**
- **Cohesion:** Degree to which elements WITHIN a module belong together. **HIGH cohesion = GOOD.**

(b) Coupling Types (LOW to HIGH — prefer LOW) ★

Type	Description	Good/Bad
Data	Modules share only data parameters	✓ Best (lowest)
Stamp	Share data structure (whole record) but use only part	Good
Control	One module controls another via flags	Moderate
External	Modules share externally imposed data format/protocol	Worse
Common	Modules share global data	Bad
Content	One module directly modifies internals of another	✗ Worst (highest)

(Message coupling sometimes listed as best in OO — via message passing.)

Mnemonic: "Data → Stamp → Control → External → Common → Content" = "**DSCECC**" (low → high).

(c) Cohesion Types (LOW to HIGH — prefer HIGH) ★

Type	Description	Good/Bad
Coincidental	Elements unrelated (grouped by chance)	✗ Worst
Logical	Elements perform similar logical category	Bad
Temporal	Elements executed at same time (e.g., init)	Weak
Procedural	Elements follow a sequence of execution	Moderate
Communicational	Elements operate on same data	Good
Sequential	Output of one is input of next	Better
Functional	Elements contribute to single well-defined task	✓ Best (highest)

Mnemonic: "**CLTP-CSF**" = Coincidental, Logical, Temporal, Procedural, Communicational, Sequential, Functional (worst → best).

(d) Design Strategies

Strategy	Approach
Top-Down	Decompose problem from high-level to detail (stepwise refinement)
Bottom-Up	Build reusable components first, then compose
Function-Oriented	Focus on functions/procedures; uses DFD, structured design
Object-Oriented	Focus on objects encapsulating data + behavior; uses UML

(e) Software Metrics

(i) LOC (Lines of Code):

- **Advantages:** Simple, language-independent counting, widely used.
- **Disadvantages:** Penalizes good/compact code, language-dependent meaning, can't measure before coding, ignores complexity.

(ii) Halstead's Software Science:

Let n_1 = distinct operators, n_2 = distinct operands, N_1 = total operators, N_2 = total operands.

Metric	Formula
Vocabulary (n)	$n = n_1 + n_2$
Length (N)	$N = N_1 + N_2$
Volume (V)	$V = N \times \log_2(n)$
Difficulty (D)	$D = (n_1/2) \times (N_2/n_2)$
Effort (E)	$E = D \times V$
Time (T)	$T = E / 18 \text{ (seconds)}$
Estimated length ($\hat{N}$)	$\hat{N} = n_1 \cdot \log_2(n_1) + n_2 \cdot \log_2(n_2)$

(iii) Function Point (Albrecht):

Count 5 element types → apply weights → sum to get **UFP** → adjust with CAF.

Element	Low	Avg	High
External Inputs (EI)	3	4	6
External Outputs (EO)	4	5	7
External Inquiries (EQ)	3	4	6
Internal Logical Files (ILF)	7	10	15
External Interface Files (EIF)	5	7	10

UFP = Σ (count × weight)

CAF (Complexity Adjustment Factor) = 0.65 + 0.01 × Σ(F_i)

- 14 adjustment factors, each rated 0-5 → ΣF_i ∈ [0, 70]
- CAF range = [0.65, 1.35] → **±35% adjustment** around unity

FP = UFP × CAF

(iv) Cyclomatic Complexity V(G): ★ [VERY HIGH YIELD]

Three equivalent formulas:

- **V(G) = E – N + 2** (E = edges, N = nodes)
- **V(G) = P + 1** (P = predicate/decision nodes — if, while, for, case)
- **V(G) = R** (R = number of regions in planar CFG, including outer region)

Interpretation: V(G) = number of linearly independent paths ≈ minimum test cases for branch coverage.

McCabe's risk interpretation:

- 1-10: simple, low risk
- 11-20: moderate risk
- 21-50: high risk, needs refactoring
- 50+: untestable, extreme risk

(f) Diagrams

- **Control Flow Graph** (nodes = statements/decisions, edges = flow of control).
- **Design Structure Chart** (hierarchical module tree with data couples and control flags).

(g) Solved PYQ-Style Problems

Q1 (7 marks) — Cyclomatic Complexity of given code. (2024-25 Q6.b)

```

int compute_gcd(x, y)
int x, y;
{
    while (x != y) {
        if (x > y) then
            x = x - y;
        else
            y = y - x;
    }
    return x;
}

```

CFG nodes:

Node	Statement
1	function entry

```

| 2 | while (x != y) — predicate |
| 3 | if (x > y) — predicate |
| 4 | x = x - y |
| 5 | y = y - x |
| 6 | return x |

```

Edges:

1→2, 2→3 (while true), 2→6 (while false), 3→4 (if true), 3→5 (if false), 4→2, 5→2.

E = 7, N = 6, P = 2 (while, if)

Three formulas verified:

- $V(G) = E - N + 2 = 7 - 6 + 2 = 3$
- $V(G) = P + 1 = 2 + 1 = 3$
- $V(G) = R = 3$ regions in CFG ✓

Three independent paths:

1. 1 → 2 → 6 (*x equals y initially, loop skipped*)
2. 1 → 2 → 3 → 4 → 2 → 6 (*x > y path*)
3. 1 → 2 → 3 → 5 → 2 → 6 (*x < y path*)

Q2 (7 marks) — Function Point with CAF = ±35% adjustment. (2024-25 Q2.c pattern)

A project has: EI=5 (avg), EO=4 (avg), EQ=3 (low), ILF=2 (avg), EIF=1 (avg). All 14 $F_i = 3$.

Sol:

- UFP = $5 \times 4 + 4 \times 5 + 3 \times 3 + 2 \times 10 + 1 \times 7 = 20 + 20 + 9 + 20 + 7 = 76$
- $\Sigma F_i = 14 \times 3 = 42$
- CAF = $0.65 + 0.01 \times 42 = 1.07$
- **FP = $76 \times 1.07 \approx 81.3$**

Why ±35%: CAF min = 0.65 (when $\Sigma F_i = 0$) = $1.0 - 0.35$. CAF max = $0.65 + 0.01 \times 70 = 1.35 = 1.0 + 0.35$. Hence CAF adjusts UFP by up to ±35%.

Q3 (7 marks) — Halstead's metrics example.

Given a program fragment with $n_1 = 10$, $n_2 = 15$, $N_1 = 50$, $N_2 = 60$. Compute V, D, E, T.

Sol:

- $n = 25$, $N = 110$
- $V = 110 \times \log_2(25) = 110 \times 4.644 = 510.84$ bits
- $D = (10/2) \times (60/15) = 5 \times 4 = 20$
- $E = 20 \times 510.84 = 10216.8$ elementary mental discriminations
- $T = 10216.8 / 18 \approx 567.6$ seconds ≈ 9.5 minutes

UNIT IV — Software Testing

(a) Core Definitions

- **Testing:** Process of executing software with intent of finding errors. *Testing cannot prove absence of defects, only their presence.*
- **Error:** Human action producing incorrect result.
- **Fault (Defect/Bug):** Manifestation of error in code.
- **Failure:** Deviation of observed behavior from specified behavior (when the fault is executed).
- **Verification vs Validation** — see Unit II.
- **Test Case:** Set of inputs, execution conditions, and expected results.
- **Stub:** Dummy module called by the module under test (top-down).
- **Driver:** Dummy module that calls the module under test (bottom-up).

(b) Testing Levels

Level	Who tests	What is tested	Technique
Unit Testing	Developer	Individual module/function	White box
Integration Testing	Developer/Tester	Interactions between modules	Top-down (stubs) or Bottom-up (drivers)
System Testing	Test team	Complete integrated system	Black box
Acceptance Testing	End user	Meets user requirements	Black box (Alpha/Beta)
Regression Testing	Any	Post-change verification	Re-run old test suite

Top-Down Integration: Start with top module, use **stubs** for not-yet-integrated lower modules.

Bottom-Up Integration: Start with lowest modules, use **drivers** to call them.

(c) Black-Box vs White-Box Testing ☆

Black-Box (Functional)	White-Box (Structural)
Tester unaware of internal code	Tester sees the code
Based on requirements/specs	Based on control flow
Techniques: BVA, Equivalence Partitioning, Decision Tables, State Transition	Techniques: Statement coverage, Branch coverage, Path coverage, Data flow
Finds missing functions, interface errors	Finds logic errors, uncovered paths

(d) Black-Box Test Case Design

(i) Equivalence Partitioning: Divide input domain into classes where all values are expected to behave similarly. Pick one representative from each class.

(ii) Boundary Value Analysis (BVA): Test at boundaries of input domains — most defects occur at edges.

- *For a range [a, b]* → test $a-1$, a , $a+1$, nominal, $b-1$, b , $b+1$.
- *$4n + 1$ formula* (standard BVA): for n inputs, $4n+1$ test cases.
- *Robust BVA* (with out-of-range): $6n + 1$ test cases.
- *Worst-case BVA* (all combinations): 5^n test cases.
- *Robust worst-case*: 7^n test cases.

(e) Static Testing

- **Code Inspection:** Formal, moderated; checklist-driven; author silent.
- **Walk-through:** Informal, author leads; peers give feedback; less formal.
- **Formal Technical Review (FTR):** Between inspection and walk-through in rigor; meeting with 3-5 reviewers, written report.

(f) Special Testing Types

- **Alpha Testing:** In-house testing by developer team or selected users at developer site.
- **Beta Testing:** At customer site; real-user conditions; developer not present.
- **Mutation Testing:** Small changes ("mutants") introduced to code; if test cases catch them, suite is good.
- **Regression Testing:** Re-run previous tests after changes to ensure nothing broke.
- **Smoke Testing:** Quick sanity check after build.
- **Sanity Testing:** Targeted subset after a minor change.

(g) Solved PYQ-Style Problems

Q1 (2 marks) — Error, Fault, Failure. (2023-24 Q1.h)

Sol: *Error* = human mistake. *Fault* = incorrect step/instruction in code (caused by error). *Failure* = deviation of observed behavior from specified (caused when fault is executed).

Q2 (10 marks — 2023-24) / (7 marks — 2024-25 style) — Test cases for prime check [1,100] using BVC, Robust, Worst-case. (2023-24 Q6.a)

Sol (single input variable, range [1, 100]):

(i) Boundary Value Analysis (BVC, 5 values):

```
| Test | Input | Expected |
|-----|-----|-----|
| T1 | 1 | Neither prime nor composite (special case) |
| T2 | 2 | Prime |
| T3 | 50 | Not prime |
| T4 | 99 | Not prime |
| T5 | 100 | Not prime |
```

(ii) Robust BVA (7 values, includes just-out-of-range):

```
| Test | Input | Expected |
|-----|-----|-----|
| T1 | 0 | Invalid (out of range) |
| T2 | 1 | Special case |
| T3 | 2 | Prime |
| T4 | 50 | Not prime |
| T5 | 99 | Not prime |
| T6 | 100 | Not prime |
| T7 | 101 | Invalid (out of range) |
```

(iii) Worst-case BVA: For a single variable, worst case = same 5 values ($5^1 = 5$). For multiple inputs, 5^n combinations. Since this program has 1 input, worst-case BVA matches regular BVA.

Q3 (7 marks) — Quadratic Roots, input (a,b,c) ∈ [0,10]. BVA + EP test cases. (2024-25 Q6.a)

Equivalence Classes:

Class	Condition	Output
E1	a = 0	Not a quadratic
E2	a ≠ 0, b² − 4ac > 0	Real distinct roots
E3	a ≠ 0, b² − 4ac = 0	Equal roots
E4	a ≠ 0, b² − 4ac < 0	Imaginary roots
E5	Any input < 0 or > 10	Invalid input

EP Test Cases (1 per class):

Test	(a, b, c)	Expected
T1	(0, 5, 3)	Not quadratic
T2	(1, 5, 6)	Real roots (disc = 1)
T3	(1, 2, 1)	Equal roots (disc = 0)
T4	(1, 2, 5)	Imaginary (disc = −16)
T5	(11, 0, 0)	Invalid

Boundary Value Analysis (4n+1 = 13 test cases for n=3 variables):

Nominal = (5, 5, 5). For each variable, vary through {0, 1, 5, 9, 10} while keeping others nominal.

Test	(a, b, c)	Expected
T1 (nominal)	(5, 5, 5)	disc = 25−100 < 0 → Imaginary
T2 (a=min)	(0, 5, 5)	Not quadratic
T3 (a=min+1)	(1, 5, 5)	disc = 25−20 > 0 → Real
T4 (a=max−1)	(9, 5, 5)	disc = 25−180 < 0 → Imaginary
T5 (a=max)	(10, 5, 5)	disc = 25−200 < 0 → Imaginary
T6 (b=min)	(5, 0, 5)	disc = −100 → Imaginary
T7 (b=min+1)	(5, 1, 5)	disc = 1−100 → Imaginary
T8 (b=max−1)	(5, 9, 5)	disc = 81−100 → Imaginary
T9 (b=max)	(5, 10, 5)	disc = 100−100 = 0 → Equal roots
T10 (c=min)	(5, 5, 0)	disc = 25 > 0 → Real
T11 (c=min+1)	(5, 5, 1)	disc = 5 > 0 → Real
T12 (c=max−1)	(5, 5, 9)	disc = 25−180 → Imaginary
T13 (c=max)	(5, 5, 10)	disc = 25−200 → Imaginary

UNIT V — Software Maintenance & Project Management

(a) Core Definitions

- **Software Maintenance:** Modification of software after delivery to correct faults, improve performance, adapt to changed environment.
- **Software Evolution:** Software must continuously evolve to remain useful (Lehman's laws).
- **Re-engineering:** Examining existing software and rebuilding it to improve quality, maintainability, or performance.
- **Reverse Engineering:** Analyzing existing software to extract design/specification from implementation (code → design → SRS).
- **SCM (Software Configuration Management):** Discipline of managing changes to software artifacts throughout lifecycle.
- **CASE (Computer-Aided Software Engineering) Tools:** Software tools supporting SE activities (analysis, design, coding, testing, PM).

(b) Maintenance Categories ☆ [HIGH FREQUENCY]

Type	What	Trigger	% of effort
Corrective	Fix defects found after delivery	Bug reports	~20%
Adaptive	Adapt to new environment (OS upgrade, hardware change)	Environment change	~25%
Perfective	Add new functionality, improve performance	User feedback, enhancement requests	~50% ↑ largest
Preventive	Improve maintainability, prevent future issues	Proactive quality	~5%

Mnemonic: "CAPP" — Corrective, Adaptive, Perfective, Preventive.

(c) Reverse Engineering vs Re-engineering

Reverse Engineering	Re-engineering
Goal: UNDERSTAND existing software	Goal: REBUILD existing software
Output: specifications, design docs	Output: improved new version
Abstracts code upward	Restructures and rewrites
Non-destructive (original untouched)	Changes the product

(d) COCOMO Model (Boehm, 1981) ★

Basic COCOMO: Effort in Person-Months (PM) and Time in months.

Mode	Description	a	b	c	d
Organic	Small team, familiar environment, simple	2.4	1.05	2.5	0.38
Semi-detached	Medium team, mixed experience	3.0	1.12	2.5	0.35
Embedded	Complex, tight constraints, real-time	3.6	1.20	2.5	0.32

Formulas:

$$\text{Effort (E)} = a \times (\text{KLOC})^b \quad \text{person-months}$$

$$\text{Development time (D)} = c \times E^d \quad \text{months}$$

$$\text{Average staff size} = E / D \quad \text{persons}$$

$$\text{Productivity} = \text{KLOC} / E \quad \text{KLOC per PM}$$

Person-Month (PM): Work done by one person in one month (~152 working hours).

Three levels of COCOMO:

1. **Basic** — rough quick estimate.
2. **Intermediate** — includes 15 cost drivers (EAF = effort adjustment factor).
3. **Detailed** — Intermediate + phase-wise effort distribution.

(e) Risk Management (RMMM) — 4-step

1. **Identification** — checklist of risk categories (product size, business, customer, process, technology, development environment, staff).
2. **Projection (Assessment)** — probability × impact matrix.
3. **Mitigation, Monitoring, and Management (RMMM plan).**
4. **Contingency planning.**

(f) Configuration Management Activities

1. **Configuration Identification** — identify configuration items (CIs).
2. **Version Control** — manage multiple versions (SVN, Git).
3. **Change Control** — formal process: request → review → approve → implement → test → release.
4. **Configuration Auditing** — verify correctness.
5. **Status Reporting** — inform stakeholders.

(g) Solved PYQ-Style Problems

Q1 (2 marks) — Define Software Re-engineering. (2024-25 Q1.g, 2023-24 Q2.e)

Sol: Process of examining, understanding, and restructuring existing legacy software to improve quality, maintainability, or performance — typically via reverse engineering → restructuring → forward engineering.

Q2 (7 marks) — COCOMO numerical (predicted for 2025-26).

A software project is estimated at **60 KLOC** and developed in **organic mode**. Calculate effort, development time, average staff, and productivity.

Sol:

- $E = 2.4 \times (60)^{1.05} = 2.4 \times 71.25 = \mathbf{171.0 \text{ person-months}}$
- $D = 2.5 \times (171.0)^{0.38} = 2.5 \times 6.84 = \mathbf{17.10 \text{ months}}$
- $\text{Staff} = E / D = 171.0 / 17.10 \approx \mathbf{10 \text{ persons}}$
- $\text{Productivity} = 60 / 171.0 \approx \mathbf{0.351 \text{ KLOC/PM} \approx 351 \text{ LOC/PM}}$

Q3 (7 marks) — Categories of maintenance + cost distribution + role of Reverse Engineering. (2024-25 Q7.b)

Sol:

- 4 categories (CAPP table above).
 - **Cost distribution:** Perfective 50% > Adaptive 25% > Corrective 20% > Preventive 5%.
 - **Role of Reverse Engineering in maintenance:**
 - Recovers lost design documentation of legacy systems.
 - Identifies hidden dependencies before making changes.
 - Aids in re-engineering decision (keep, patch, or rebuild).
 - Supports migration to new platforms/languages.
 - Essential when original developers are unavailable and source code is the only artifact.
-

3. HIGH-PROBABILITY QUESTIONS (PYQ-BASED)

#	Question	Likely Section	Answer Skeleton
1	Discuss software characteristics	A or B	List 5 key characteristics → explain each briefly → example
2	Compare SDLC models (Waterfall vs Spiral vs Prototype)	C-Q3	Intro → 4-column comparison table → one diagram each → "when to use"
3	Explain Software Quality Attributes / McCall's model	C-Q3	3 categories → 11 attributes → diagram of triangle
4	Requirement Elicitation techniques	B or C-Q4	6-step RE process → 6 elicitation techniques → 1 line each
5	Feasibility Study — types and importance	B	TOE (+ Legal + Schedule) → 1 para each → importance bullet
6	Draw DFD (Library/Hotel/ATM) with 2 levels + STD	C-Q4	Context diagram → Level 1 → Level 2 (one sub-process) → STD of one flow
7	SEI CMM vs ISO 9000 + 5 CMM levels	C-Q4 or C-Q5	Comparison table → 5 levels (IRDMO) with description → conclusion
8	All 9 UML diagrams (static + dynamic)	C-Q4	Static: Class, Object, Component, Deployment, Package. Dynamic: Use Case, Sequence, Collaboration, Activity, State
9	Explain Cohesion and Coupling + all types	C-Q5	Definitions → 7 coupling types (low→high) → 7 cohesion types (low→high) → design rule
10	Function Point / CAF / UFP calculation	B or C-Q5	Formula → 5 element counting table → CAF calc → ±35% proof
11	Cyclomatic Complexity of given CFG/code	C-Q6	Draw CFG → count E, N, P, R → verify all 3 formulas → list independent paths
12	BVA + EP test cases (Prime/Quadratic/Triangle)	C-Q6	EP classes → EP test cases → BVA boundary values → BVA test cases
13	White Box vs Black Box testing	B or C-Q6	Definitions → comparison table → 3 techniques each
14	Unit/Integration/System testing with stubs/drivers	B	Level table → top-down vs bottom-up → stub vs driver → when each used
15	Categories of software maintenance + cost	B or C-Q7	CAPP table → cost % → "software as evolutionary entity"
16	COCOMO model + Person-Month	C-Q7	3 modes + constants → formulas → numerical example
17	Reverse Engineering vs Re-engineering	A or C-Q7	Comparison table → role in maintenance → example
18	CASE tools — structure + benefits	C-Q7	Definition → categories (upper/lower/integrated) → tool examples → benefits

3.5 🎯 ACTUAL AKTU PYQ BANK — FULLY SOLVED (2023-24 & 2024-25)

This is the highest-signal section. These are the exact questions that appeared.

📖 AKTU KCS 601 — 2023-24 Paper (Paper ID 410299, 100 marks)

SECTION A (2 × 10 = 20):

Q	Question	Model 2-line Answer
a	Define Software Engineering	Systematic, disciplined, quantifiable engineering approach to software development, operation, and maintenance (IEEE).
b	Software characteristics	Intangible, doesn't wear out, engineered (not manufactured), custom-built, logical.
c	Need of SRS	Acts as contract between client-developer, reduces miscommunication, baseline for design/testing.
d	Stubs and Drivers	<i>Stub</i> = dummy called by module under test (top-down). <i>Driver</i> = dummy that calls module (bottom-up).
e	LOC + 2 pros/cons	<i>Pros</i> : simple, universal. <i>Cons</i> : penalizes compact code, language-dependent.
f	Pseudo Code vs Algorithm	Pseudo code uses programming-like syntax; algorithm is language-neutral step-by-step logic.
g	White box testing importance	Tests internal logic, finds untested branches, ensures code coverage.
h	Error, Fault, Failure	Human error → code fault → runtime failure.
i	Reasons for rising software costs	Growing complexity, maintenance burden, skilled personnel shortage.
j	Need of Risk Management	Proactive identification of threats to schedule/cost/quality; prevents project failure.

SECTION B (10 × 3 = 30) — Attempt 3 of 5:

- a) "SE is layered technology" → 4 layers (Tools on top, then Methods, Process, Quality Focus at base).
- b) Feasibility Study importance + types → TOE (Technical, Operational, Economic) + Legal, Schedule.
- c) Code Inspection vs FTR vs Walk-through → comparison table.
- d) Mutation / Alpha & Beta / Regression testing → short note on each.
- e) Software Re-engineering → definition + purpose + typical process (reverse eng → restructure → forward eng).

SECTION C (10 × 5 = 50) — One of 2 per unit:

Q	Part A	Part B
Q3	Software Quality Attributes (detail)	SDLC + activities in SDLC
Q4	Requirement Elicitation techniques	SEI CMM vs ISO 9000 + 5 CMM levels ★
Q5	Cohesion & Coupling + all forms ★	Software metric + size estimation metrics
Q6	Prime check [1,100] — BVA, Robust, Worst-case test cases ★	Integration Testing + approaches
Q7	Maintenance need + categories	COCOMO + Person-Month

AKTU BCS 601 — 2024-25 Paper (Paper ID 250535, 70 marks) — FULLY SOLVED

SECTION A (2 × 7 = 14)

Q1.a — Components of software with brief description (K1).

(See Unit I (a).) 3 components: Programs + Documentation + Operating procedures.

Q1.b — Justify "Software does not wear out and is not manufactured" (K2).

(See Unit I Solved Q1.)

Q1.c — Software Risk Analysis Management (K1).

(See Unit II Solved Q1.)

Q1.d — Define Use Case + template (K3).

(See Unit II Solved Q2.)

Q1.e — Size-Oriented Measures (K2).

Size-oriented metrics count product "size" to normalize productivity/quality:

- **LOC** (Lines of Code) — most common; count of source statements.
- **KLOC** (1000 LOC).
- **DLOC / DSI** (Delivered LOC / Delivered Source Instructions).
- Derived measures: Productivity = KLOC / PM; Quality = defects / KLOC; Cost = \$ / LOC.
- *Pros*: simple, universal. *Cons*: language-dependent, doesn't account for complexity, penalizes concise code.

Q1.f — Justify "Exhaustive Testing is not possible" (K5).

- *Combinatorial explosion*: A 10-bit input has $2^{10} = 1024$ cases; real programs have billions of input combinations.
- *Infinite paths*: Loops create potentially infinite execution paths.
- *Time & cost*: Testing all paths = infeasible.
- *Conclusion*: Testing only provides confidence (not proof) — hence **risk-based, coverage-based selection** (BVA, EP, path coverage) is used instead.

Q1.g — Define Software Re-engineering (K3).

Examining and rebuilding existing software to improve quality/maintainability without changing its functionality; involves reverse engineering → restructuring → forward engineering.

SECTION B (7 × 3 = 21)

Q2.a — Software quality attributes + SQA procedure + Error Index derivation (K2).

Quality attributes: McCall's 11 factors in 3 categories (see Unit I).

SQA procedure (Statistical Quality Assurance):

1. Classify defect data by root cause (logic, interface, data, etc.).
2. Identify *vital few* (Pareto — 80/20).
3. Trace causes to underlying process issues.
4. Correct process.
5. Monitor via control charts.

Error Index (EI) formula:

$$EI = \sum_{i=1}^n (i \times PI_i) / (PS)$$

where i = phase index, PI_i = defects found in phase i (weighted by severity), PS = total product size (e.g., KLOC). Lower EI → better quality.

Q2.b — Library Management DFD (2-level) + STD (K3).

(See Unit II Solved Q3.)

Q2.c — LOC drawbacks + Albrecht FP + CAF adjusts ±35% (K4).

LOC drawbacks:

1. Language-dependent.
2. Penalizes expressive languages.
3. Can't estimate before coding.
4. Ignores complexity.
5. Copy-paste inflates LOC.

Albrecht FP Count Method:

Count 5 element types (EI, EO, EQ, ILF, EIF) × weights → **UFP**. Apply **CAF** to get FP.

Proof CAF ∈ [0.65, 1.35] = unity ±35%:

- 14 general system characteristics, each $F_i \in [0, 5] \rightarrow \sum F_i \in [0, 70]$.
- $CAF = 0.65 + 0.01 \times \sum F_i$
- Min CAF = $0.65 + 0 = 0.65 = 1.0 - 0.35$
- Max CAF = $0.65 + 0.70 = 1.35 = 1.0 + 0.35$

∴ CAF adjusts UFP by **±35%** around unity. **Q.E.D.**

Q2.d — Unit / Integration / System testing (K2).

(See Unit IV (b) testing levels table.)

Q2.e — "Software maintenance is inevitable" + types + cost (K2).

Justification: Software must evolve — environment changes (new OS, hardware), requirements shift (user feedback, market), defects emerge in production. **Lehman's Laws** confirm: "A program undergoing continuing change tends to grow in complexity unless work is done to reduce it."

Types + relative cost: CAPP table — Perfective (50%) > Adaptive (25%) > Corrective (20%) > Preventive (5%).

SECTION C (7 × 5 = 35) — FULLY SOLVED

Q3.a — Spiral Model + diagram + selection criteria (K3).

(See Unit I Solved Q3.)

Q3.b — Define software + SE + components + crisis (K4).

- *Software* = programs + data + documentation + operating procedures.
- *Software Engineering* = IEEE definition.
- *Components* = Programs, Documentation, Operating procedures (triad).
- *Software crisis causes:* growing hardware capability outpacing software dev, increasing complexity, lack of skilled engineers, no standardized processes. Symptoms: over-budget, late delivery, poor quality, unmaintainable legacy.

Q4.a — Requirements Engineering Process + Use Case template + example (K3).

(See Unit II (b) + Unit II Solved Q2.)

Example (ATM Withdraw Cash):

- **Use Case:** Withdraw Cash

- **Actor:** Bank Customer
- **Precondition:** Valid card inserted, correct PIN entered.
- **Main flow:** Select Withdrawal → Enter amount → System verifies balance → Dispenses cash → Prints receipt.
- **Alternate flow:** Insufficient funds → Display error.
- **Postcondition:** Balance updated.

Q4.b — 9 UML diagrams (static + dynamic) (K4).

Category	Diagram	Purpose
Static (Structural)	Class	Classes, attributes, methods, relationships
	Object	Instances at a point in time
	Component	Physical components and dependencies
	Deployment	Hardware nodes and artifacts
	Package	Grouping of related elements
Dynamic (Behavioral)	Use Case	Actor-system interactions
	Sequence	Time-ordered message flows
	Collaboration (Communication)	Structural org. of interacting objects
	Activity	Workflow / control flow
	State (Statechart)	State transitions of an object

(UML 2.0 added a 10th — Timing diagram; older syllabi list 9.)

Q5.a — Attributes of good software design + Coupling & Cohesion measures (K2).

Good design attributes:

1. Correctness (implements requirements)
2. Completeness
3. Efficiency
4. Flexibility (easy to modify)
5. Consistency (uniform style)
6. **Low coupling, high cohesion**
7. Understandability
8. Modularity

Coupling measures strength of inter-module dependence (Low = Good).

Cohesion measures strength of intra-module relatedness (High = Good).

(See Unit III (b) and (c) tables.)

Q5.b — SEI CMM vs ISO 9000 with distinctive features (K2).

(See Unit II (e) table.)

Q6.a — Quadratic Roots BVA + EP test cases. (K5)

(See Unit IV Solved Q3 — full 5-class EP + 13-case BVA.)

Q6.b — CFG + Cyclomatic Complexity for compute_gcd. (K4)

(See Unit III Solved Q1 — $V(G) = 3$, 3 independent paths.)

Q7.a — CASE development environment structure + benefits (K2).

Structure (layers):

1. **Hardware platform** — workstations, servers.
2. **OS / database layer** — common OS, repository DB.
3. **Portability services** — interface between tools and environment.
4. **Integration framework** — tool-to-tool communication.
5. **Tools layer** — upper CASE (analysis/design), lower CASE (coding/testing), integrated CASE (I-CASE).

Categories of CASE tools:

- Upper CASE: analysis, design (e.g., Rational Rose, Visio, Enterprise Architect)
- Lower CASE: coding, testing (e.g., Eclipse, JUnit)
- Integrated CASE (I-CASE): end-to-end (e.g., IBM Rational Suite)
- Project Management CASE (e.g., MS Project, JIRA)

Benefits:

1. Improved productivity & consistency.
2. Standardized documentation.
3. Error reduction through automation.
4. Better communication among team.
5. Reusable components and templates.

6. Faster development cycle.

Q7.b — Maintenance categories + cost distribution + Reverse Engineering. (K3)

(See Unit V Solved Q3.)

🔗 TOP PREDICTIONS FOR 2025-26 BASED ON THESE TWO PAPERS

Appeared in BOTH years (near-certain repeat):

- ✓ Software characteristics (2-mark)
- ✓ Error/Fault/Failure (2-mark)
- ✓ Re-engineering definition (2-mark)
- ✓ **SEI CMM vs ISO 9000** (Section C) ★
- ✓ **Cohesion & Coupling** (Section C) ★
- ✓ Integration testing + stubs/drivers
- ✓ Maintenance categories + cost
- ✓ **BVA / EP test case design** ★

In 2024-25 (new pattern, likely to stick):

- ✗ Use Case template (2-mark)
- ✗ Function Point + CAF = $\pm 35\%$ numerical
- ✗ Cyclomatic Complexity + CFG for given code
- ✗ UML diagrams (9 types)
- ✗ DFD + State Transition Diagram
- ✗ "Justify" K5-level questions
- ✗ Statistical SQA / Error Index formula

From 2023-24 (due to return):

- ⚙ SDLC model comparison (in-depth)
- ⚙ Software Quality Attributes (McCall's 11)
- ⚙ Feasibility Study (TOE)
- ⚙ Code Inspection vs Walkthrough vs FTR
- ⚙ Mutation / Alpha / Beta / Regression testing
- ⚙ Re-engineering process in detail

Probable 2025-26 additions (syllabus but not yet tested recently):

- ⚠ **COCOMO numerical** ★ — both years asked theoretically; a calculation is overdue. HIGH odds.
- ⚠ **Halstead's Software Science** numerical ($n_1, n_2, N_1, N_2 \rightarrow V, D, E, T$)
- ⚠ ER Diagram / Decision Table design
- ⚠ Software Configuration Management (SCM) process
- ⚠ RMMM plan / risk matrix
- ⚠ Architecture design styles (layered, client-server, MVC)

4. CONSOLIDATED FORMULA SHEET

Topic	Formula	When to Use
Cyclomatic Complexity	$V(G) = E - N + 2$	Edges & Nodes method
Cyclomatic Complexity	$V(G) = P + 1$	Predicate nodes method
Cyclomatic Complexity	$V(G) = R$	Regions in planar CFG
Halstead Vocabulary	$n = n_1 + n_2$	Operators + operands
Halstead Length	$N = N_1 + N_2$	Total operators + operands
Halstead Volume	$V = N \times \log_2(n)$	Program size in bits
Halstead Difficulty	$D = (n_1/2) \times (N_2/n_2)$	Program effort difficulty
Halstead Effort	$E = D \times V$	Mental discrim. required
Halstead Time	$T = E / 18$ (seconds)	Programmer time estimate
Halstead Estimated Length	$\hat{N} = n_1 \log_2 n_1 + n_2 \log_2 n_2$	Prediction from vocabulary
UFP	Σ (count $\times$ weight) over 5 element types	Function Point
CAF	$0.65 + 0.01 \times \Sigma F_i$	14 adjustment factors
FP (adjusted)	$FP = UFP \times CAF$	Adjusted function point
CAF range	$[0.65, 1.35] = 1 \pm 0.35$	$\pm 35\%$ adjustment proof
COCOMO Effort	$E = a \times (KLOC)^b$	Person-months
COCOMO Time	$D = c \times E^d$	Development months
COCOMO Staff	$N = E / D$	Average persons
Productivity	$KLOC / E$	Lines per person-month
Quality	defects / KLOC	Defect density
BVA test cases	$4n + 1$ (standard)	n = number of inputs
Robust BVA	$6n + 1$	Includes just-out-of-range
Worst-case BVA	5^n	All combinations
Robust worst-case	7^n	All combinations + invalid
Error Index	$EI = \Sigma (i \times P_i) / PS$	Statistical SQA

5. MEMORY AIDS (MNEMONICS)

Concept	Mnemonic
5 SDLC phases (Waterfall)	" RDISM " — Requirement, Design, Implementation, Testing (integration), Maintenance
4 Spiral quadrants	" O-R-D-P " — Objectives, Risk analysis, Development, Planning
3 software components	" PDO " — Programs, Documentation, Operating procedures
Software characteristics	" I-W-E-L-C " — Intangible, doesn't Wear out, Engineered not manufactured, Logical, Custom-built
McCall quality categories	" O-R-T " — Operation, Revision, Transition
11 McCall attributes	<i>Operation</i> : CREIU — Correctness, Reliability, Efficiency, Integrity, Usability. <i>Revision</i> : MFT — Maintainability, Flexibility, Testability. <i>Transition</i> : PRI — Portability, Reusability, Interoperability
5 CMM Levels	" IRDMO " — Initial, Repeatable, Defined, Managed, Optimizing
Feasibility Study types	" TOELS " — Technical, Operational, Economic, Legal, Schedule
RE Process phases	" IEEN-SV " — Inception, Elicitation, Elaboration, Negotiation, Specification, Validation
Elicitation techniques	" I-Q-B-J-F-P " — Interviews, Questionnaires, Brainstorming, JAD, FAST, Prototyping
Coupling (low→high)	" DSCECC " — Data, Stamp, Control, External, Common, Content
Cohesion (low→high)	" CLTP-CSF " — Coincidental, Logical, Temporal, Procedural, Communicational, Sequential, Functional
Testing levels	" UISA-R " — Unit, Integration, System, Acceptance, Regression
Maintenance types	" CAPP " — Corrective, Adaptive, Perfective, Preventive
Maintenance cost order	"Perfective is half , Adaptive a quarter , Corrective one-fifth , Preventive a twentieth " (50/25/20/5)
COCOMO modes	" Organic → Semi → Embedded " — small/simple → medium → complex/real-time
White box test coverage	" SBPL " — Statement, Branch, Path, Loop coverage
Black box techniques	" BED-S " — BVA, Equivalence partitioning, Decision tables, State-based
Stubs vs Drivers	"Stub is beneath (top-down), Driver is above (bottom-up)"
UML 9 diagrams	<i>Static</i> : "COCPD" — Class, Object, Component, Package, Deployment. <i>Dynamic</i> : "USCAS" — Use case, Sequence, Collaboration, Activity, State

Visual hooks:

- **Spiral model**: Think of a watch spring — keeps spiraling outward, each loop bigger and costlier.
- **Waterfall**: Water only flows DOWN — once you pass a stage, no going back.
- **Coupling vs Cohesion**: *Coupling* = how close two magnets are (less = better); *Cohesion* = how tightly glued one module's parts are (more = better).
- **BVA**: Test where fences meet — bugs hide at boundaries.
- **Stub vs Driver**: Stubs stub-out the callees below; Drivers drive the callers from above.

6. COMMON MISTAKES & EXAMINER TRAPS

1. **Confusing stubs and drivers** — Stub is called BY the module under test (replaces callees, used in top-down). Driver CALLS the module under test (replaces caller, used in bottom-up). Getting this backwards = instant mark loss.
2. **Mixing Verification and Validation** — Verification = "building right" (static, internal). Validation = "right product" (dynamic, external). Use V&V in that order.

3. **Wrong order of coupling/cohesion** — Content is WORST coupling (not best). Functional is BEST cohesion (not worst). Draw the ladder carefully.
4. **Calling Cyclomatic Complexity "line count"** — It's a path-count metric, not size. Always draw the CFG.
5. **Not drawing the CFG** — Every cyclomatic complexity question requires a drawn CFG with labeled nodes and edges. No CFG = 50% marks lost.
6. **COCOMO mode mismatch** — Organic = small team, simple. Semi-detached = medium. Embedded = real-time/critical. Don't use embedded constants for a simple CRUD app.
7. **Forgetting units in COCOMO** — Effort in person-months, Time in months, Staff in persons, LOC in KLOC (1000 LOC = 1 KLOC).
8. **BVA test case count** — Single input range gives 5 test cases (a, a+1, nominal, b-1, b). For n inputs: **$4n+1$** , not $5n$.
9. **Missing "nominal" in BVA** — The formula $4n+1$ includes the single nominal case. Forgetting it → answer off by 1.
0. **Error Index formula** — Pl_i is weighted defect count per phase, not just count. Weight = severity.
1. **Function Point weight confusion** — ILF weight $\neq$ EIF weight. ILF (internal, heavier weights 7/10/15). EIF (external, lighter 5/7/10). Don't swap.
2. **CAF range** — It's [0.65, 1.35], NOT [0, 1]. The $\pm 35\%$ is relative to unity (1.0), not absolute.
3. **Waterfall "feedback arrows"** — Classical waterfall has NO feedback. The "modified waterfall" allows backtracking. State which version you're drawing.
4. **Spiral $\neq$ Iterative** — Spiral is a specific iterative model with **risk analysis per loop**. Don't just say "iterative."
5. **ISO 9000 $\neq$ ISO 9001** — ISO 9000 is the family; ISO 9001 is the specific certifiable quality management standard.
6. **CMM "assessment" vs "certification"** — CMM is *assessed*, ISO is *certified*. Wrong word = mark deducted.
7. **Use Case $\neq$ Scenario** — A Use Case is a set of scenarios (happy path + alternates + exceptions). A single scenario is ONE execution path.
8. **DFD rules violated** — Never show data flow directly between two external entities, or between two data stores. Every process must have at least one input AND one output.
9. **UML 9 or 14 diagrams?** — Old syllabus = 9. UML 2.0 = 14. AKTU syllabus says 9 — stick with it unless asked otherwise.
0. **Maintenance % sum not 100%** — Perfective 50 + Adaptive 25 + Corrective 20 + Preventive 5 = 100. Some students write 40/30/25/5 or similar — stick with 50/25/20/5.
1. **LOC vs SLOC vs DLOC confusion** — LOC = any line; SLOC = executable source lines (excludes blanks/comments); DLOC = delivered LOC. State which you're using.
2. **Alpha/Beta site** — Alpha = developer site. Beta = customer site. Don't reverse.
3. **Regression $\neq$ Retesting** — Retesting verifies a fixed defect. Regression checks that the fix didn't break something else. Both required after a change.
4. **Reverse engineering is NOT illegal/hacking** — It's a legitimate SE activity for legacy code analysis, maintenance, and migration.
5. **Skipping the diagram for UML/DFD questions** — Text-only answers for diagram questions get max 40% of the marks. Always draw.

7. LAST-24-HOUR REVISION CHECKLIST (Top 20)

1. ✓ **Cyclomatic Complexity** — 3 formulas + one CFG drawn from code.
2. ✓ **Function Point** — UFP calculation + $CAF = 0.65 + 0.01 \sum F_i$, $\pm 35\%$ proof.
3. ✓ **COCOMO** — 3 modes, formulas E and D, one numerical.
4. ✓ **BVA + EP** — $4n+1$ formula + one worked example (prime or quadratic).
5. ✓ **Coupling types (low→high)** — DSCECC.
6. ✓ **Cohesion types (low→high)** — CLTP-CSF.
7. ✓ **SDLC models comparison** — especially Spiral (risk analysis).
8. ✓ **SEI CMM 5 levels** — IRDMO mnemonic.
9. ✓ **ISO vs CMM** — comparison table.
0. ✓ **Maintenance types + cost %** — CAPP, 50/25/20/5.
1. ✓ **Reverse vs Re-engineering** — table.
2. ✓ **Stubs vs Drivers** — which for top-down vs bottom-up.
3. ✓ **McCall's 11 quality attributes** — 3 categories.
4. ✓ **RE Process 6 phases** — IEEN-SV.
5. ✓ **Elicitation techniques** — 6 methods.
6. ✓ **DFD rules + 2-level structure**.
7. ✓ **UML 9 diagrams** — static 5 + dynamic 4.
8. ✓ **Alpha vs Beta testing**.
9. ✓ **Halstead's metrics** — 6 formulas (n, N, V, D, E, T).

0. ✓ **Error/Fault/Failure** chain.

"One-of-each" must-not-forget list

Unit	1 Formula	1 Diagram	1 Concept
I	$E = a(KLOC)^b$ (COCOMO)	Spiral Model (4 quadrants)	Software is engineered, not manufactured
II	$EI = \sum(i \cdot PI_i) / PS$	DFD Level-0 + Level-1	CMM 5 levels (IRDMO)
III	$V(G) = E - N + 2 = P + 1 = R$	Control Flow Graph	Low coupling + High cohesion
IV	BVA: $4n + 1$ test cases	Top-down integration (stubs)	White box $\neq$ Black box techniques
V	$D = c \cdot E^d$ (COCOMO time)	RMMM risk matrix	Perfective 50% > Adaptive 25% > Corrective 20% > Preventive 5%

8. ANSWER-WRITING STRATEGY FOR 100% MARKS

Structure for a 7-Mark Answer (AKTU Section B/C)

1. Heading: Underline the question topic	0.5 marks
2. 1-2 line INTRODUCTION / Definition	1 mark
3. LABELLED DIAGRAM (for any model/DFD/CFG)	2 marks
4. FORMULA / ALGORITHM / TABLE	1.5 marks
5. WORKED EXAMPLE or CALCULATION	1.5 marks
6. CONCLUSION / "Hence" / Boxed final answer	0.5 marks

7-mark depth cue: About 1 page of content. Diagrams are **mandatory** for SE (SDLC models, DFD, CFG, UML, coupling/cohesion hierarchy).

Time Allocation — 3-Hour Paper (70 marks)

Section	Marks	Questions	Time	Per Q
A	14	7×2	20 min	3 min
B	21	3×7	40 min	13 min
C	35	5×7	100 min	20 min
Buffer/Review	—	—	20 min	—

Golden rule: Section C first (35 marks, 5 questions from 5 units — no skipping). Then A (guaranteed 14). Then B (pick easiest 3 of 5).

AKTU Presentation Checklist

- ✓ **Underline** headings and key terms.
- ✓ **Always draw the diagram** for SDLC models, DFD, CFG, UML, structure charts.
- ✓ **Label every node/edge** in CFG; name every arrow in DFD.
- ✓ **Use pencil for diagrams**, pen for text.
- ✓ **Write formula on a separate line** (e.g., " $V(G) = E - N + 2$ ") BEFORE substituting.
- ✓ **Box the final numerical answer** — $V(G) = 3$, CAF = 1.07, Effort = 171 PM.
- ✓ **Use tables for comparisons** — Coupling types, CMM levels, maintenance types.
- ✓ **Mention real examples** — Waterfall (classical banking), Spiral (DoD projects), CMM Level 5 (TCS, Infosys).
- ✓ **State assumptions** in numericals — "Assume organic mode" or "Assume average complexity weights."

AKTU Examiner Preferences (from both papers)

- Rewards **diagrams + tables** heavily.
- Loves **"Justify"** K5-style questions — always structure as *Claim → Reason 1 → Reason 2 → Conclusion*.
- Expects **real-world examples** for models/techniques.
- Numerical questions: formula → substitution → calculation → boxed answer.

Final Motivation

*"In Software Engineering, marks come from diagrams, tables, and formula-boxed answers — not from prose alone."
Master the 20 items in Section 7, solve every numerical from 2024-25, and 90%+ is within reach.*

Good luck! 